

Crop-Box-Free Calibration-Board Detection in VLP-32C LiDAR Point Clouds

Phase 7 — Projection, Geometry, and a Learned Segmenter

LCTK (LiDAR and Camera Toolkit) — Internal Engineering Report

Experiment: experiments/board-detection-2d · 2026-07

Abstract — The current LCTK calibration pipeline localizes the target board with a hand-tuned crop box that must be re-tuned per scene, then runs RANSAC/ICP inside it. This phase asked whether the board — a ~ 1 m diamond, plain or hollow — can be found **anywhere** in a VLP-32C cloud with only its geometry as a prior, no manual ROI, and no human in the loop. We pursued two lines. (1) A **geometry** pipeline (plane-fit → orthographic 2D projection → OpenCV quad/square fit) with three candidate generators and a shared scorer, hardened across eight stages. It reaches a usable single-frame operating point — **49.3% recall at 93.0% precision** (stage 6), or **44.1% recall at 100% precision** (stage 8, isolation gate) — but hits a **structural recall ceiling** near 44–49%: single-frame geometry cannot **select** a sparse, ring-gapped board over compact, board-sized clutter. (2) A **learned** segmenter. To train it we built a faithful ray-based VLP-32C simulator (casting the sensor’s real 32 beam angles) and a ~ 0.21 M-parameter U-Net (BoardUNet). Trained purely on synthetic range images (val IoU 0.99) and evaluated on 535 real frames, it reaches **99.3% recall** — **breaking the geometry ceiling by 2×** — but initially only **14.8% precision**, firing on real free-standing fixtures absent from the synthetic clutter distribution. A single retrain on enriched synthetic clutter (no eval-side or inference-time change) lifted precision to **66.1%** at zero recall cost and cut latency 3×. The honest bottom line: one cheap forward pass now gives ~ 99% recall / 66% precision; recall is solved, precision is improved 4.4× but is **not yet** at geometry’s 93–100%. The eight geometry stages built the precision tools; the CNN supplies the recall; the path to high-on-both is further clutter enrichment (or a composite pipeline), which this phase surfaces but does not build.

Contents

1	Introduction and Problem Statement	3
2	Geometry-Based Detection	3
2.1	The pipeline	3
2.2	Climbing the recall/precision curve	4
3	A Ray-Based VLP-32C Simulator	7
3.1	Sensor model	7
3.2	Analytic primitives and ray casting	7
3.3	The shared range-image renderer (the train/eval linchpin)	7
3.4	Domain-randomized scene generation	8
3.5	Fidelity: the aliasing fix, verified	8
4	The CNN Detector	9
4.1	Architecture: BoardUNet	9
4.2	Training and synthetic validation	9
4.3	Synth → real transfer: the decisive test	10
5	Hybrid Attempt and the Precision Fix	11

5.1	Why the obvious hybrid failed	11
5.2	The fix: enrich the synthetic clutter, retrain	11
5.3	Result: $v1 \rightarrow v2$	11
6	Consolidated Results	13
7	Discussion and Future Work	13
8	Conclusion	14

1 Introduction and Problem Statement

The single most operator-hostile step of an LCTK calibration session is the **crop box** (`bbox.json5`): a hand-tuned 3D region of interest that isolates the calibration board before RANSAC/ICP fitting can run. The box must be re-tuned for every new scene — a dedicated tool (`filter_box_tuner`) exists solely to ease that pain — and the ICP stage inside it needs a reasonable initial pose and ~ 100 ms/frame, with the crop box doing most of the work of isolating the board.

Goal. A detector that finds the board anywhere in the scene, in real time (~ 100 ms/frame budget at 10 Hz), with only board geometry as a prior — removing the crop box and the per-scene setup cost. The board itself is also changing: a plain **0.5–1 m diamond without holes** (easier to fabricate and move) is under consideration, so the detector must key on the **square border only**; board size is a config parameter, holes are an optional extra cue, never a requirement. Ideally the method is sensor-generic (a ring field is never read by any algorithm, keeping the door open for solid-state LiDARs).

Constraints and data. The only recorded data in the repo is `lctk_sample_data`’s VLP-32C pcap playback, datasets 1–5, decoded once via `velodyne_decoder` and cached. Each dataset is a **single static capture** (the board does not move within a dataset). There is **no ground-truth board pose**; the legacy crop-box centre — never used by any algorithm — serves as the only sanity reference. “In-bbox” (a detection whose centre lands in that box) is used throughout as the true-board / clutter classifier, so all recall/precision numbers below are directly comparable across stages.

Two lines of attack. Stages 1–8 build and harden a **geometry** pipeline (Section 2). Stages 9–10 build a ray-based **simulator** (Section 3) and train a **CNN** segmenter (Section 4, Section 5). Section 6 consolidates the numbers; Section 7 covers what remains.

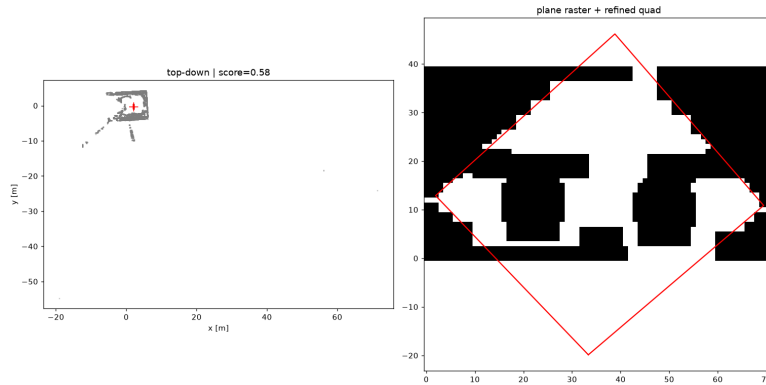


Figure 1: Geometry pipeline finding the true board (generator B, dataset 3). The scored quad is fit to the sparse, ring-gapped board patch after plane-fit and 2D projection; the raster shows the hollow-diamond outline. This is the “clean candidate” case — the projection

1. scorer core is sound whenever a board-shaped candidate reaches it.

2 Geometry-Based Detection

2.1 The pipeline

The pivotal design choice is how to map 3D points to 2D while keeping the board’s square shape intact. A survey (range image, bird’s-eye, virtual pinhole, plane-fit) settled on the one distortion-free, metric, scan-structure-agnostic option, which is also what successful published pipelines use (`velo2cam`, `lvt2calib`, `ILCC`, `ACSC`, `FAST-Calib`):

plane-fit (PCA) → **orthographic projection** into plane coordinates (pixels = metres $\times$ resolution) → **rasterize** occupancy image → morphological **close** → `cv2.findContours` → **quad fit** (`minAreaRect`) → **side-line refit** on raw projected points → **score** (side length vs config, squareness, fill ratio, edge straightness) → **pose**.

With no crop box, candidates must be generated from the full scene. Three generators feed the **same** shared scorer and were compared head-to-head:

- **A — iterative RANSAC** (velo2cam style): repeatedly RANSAC the largest plane, remove inliers, gate each patch by size. Structurally weak — the board is a **small** plane that surfaces late, mixed with coplanar clutter.
- **B — Euclidean clustering after big-plane removal**: strip only **large** planes (ground, walls), cluster the remainder, gate each cluster (flatness + size). A free-standing board forms a clean cluster; cheap and the only generator carried through to a usable result.
- **C — normal-based region growing**: grow regions of coherent normal. Handles board-against- wall but normal estimation on sparse rings is noisy and 3× over budget in Python.

First-benchmark verdict (stage 1). A never recalled the board at all (0% every dataset, even after a scientifically-correct `dist_thresh` fix that made yield **worse**). C detects board-sized objects at high rate but they are **clutter**, not the board (0/28 ds3 detections at the board location; metre-scale jitter). B found the **true** board — verified against the crop-box reference on dataset 3 — but on only ~ 2% of frames. The diagnosis that framed the rest of the phase: at ~ 2 m range the 32 rings put only a handful of stripes on the board; after voxel downsampling the board cluster is 300–600 points with plane-fit RMS **at the sensor noise floor** (0.029–0.031 m), so ring gaps fragment it and the quad fails a gate.

2.2 Climbing the recall/precision curve

Stages 2–8 attacked that bottleneck one lever at a time. The honest arc includes two negative results reported straight:

Stage / lever	What it did
2 — frame accumulation	Fixed . Static capture re-samples the same ring angles: 10 frames → 1.95× density (not late); and DBSCAN fragments the board further . Recall collapsed to 0%. nate N frames to den- sify the board past ring gaps.
2 — stance term	Fixed . Kills one of two clutter-panel orientations (18→ 0), leaves the other; does not regress the true board. grav- ity- align- ment score (di- a- mond stands on a cor- ner).
3 — anisotropic clustering	Candidate generation fixed (board-shaped candidate reaches scorer on 31% of ds3 frames vs 5%), but merged patch scores low (median 0.07). End-to-end recall a wash. Bottleneck moved to the scorer. points by a range-

Stage / lever	What it did
	scaled factor before DB-SCAN to bridge ring gaps.
4 – stripe-tolerant scorer	Grow 1.3% → 43.0% (162 true / 68 clutter). Real win for the board and a comparably-sized new false-positive problem — the same close inflates clutter fill too. Bottleneck now pure discrimination . entered anisotropic morphology: close on the fill raster; coarse quad from raw points.
5 – stance / edge / squareness gates	Distance Distance-gate retains full recall, cuts clutter 78%: 30.5% / 91.6% . --strict-diamond reaches 100% precision but at a ~ 7:1 recall cost. Residual = persistent static panels . geom-etry gates replacing the (now off-the-table) hole cue.
6 – flatness gate	Raise Pareto win: 49.3% recall / 93.0% precision . Recall nearly doubled, precision rose — flared trade did not occur (true detections grow faster than clutter). fit RMS gate 0.035 → 0.045

Stage / lever	What he did
	(a C-04-style noise-floor correction).
7 – fixed-size square fitter	Refuted on real data. Lost recall (best 40.6%) AND precision (best 85.6%), doubled latency (60 → 128 ms). Residual ranking prefers compact clutter over the sparse board; the board's ring-gapped perimeter posts a residual above the gate. full θ-sweep fixed-size fit to rescue stance-rejected frames.
8 – isolation discriminator	Recall 41% / 100% precision (clutter 20 → 0), +3.4 ms only. First 100% precision without the strict-diamond recall collapse. Holds up live , unlike stage 7. can-dates with coplanar continuation beyond the fitted edges (board is free-standing; clutter is a wall patch).

The recall ceiling is structural. Stage 7's deepest finding, reproduced at the whole- detector level in stage 8: on the stance-rejected recall population, a compact clutter candidate already **outscores** the sparse board at the

2D-score **selection** step ~ 80% of the time (the board is the frame’s own top-scoring candidate only 19.7% of the time). Board-vs-clutter discrimination is a **selection** problem present independently at all three single-frame gates (2D score, fit residual, stance floor) — no fitter design fixes it. Stance and isolation are **complementary** gates for two clutter classes (free-standing vs wall- embedded), not substitutes; dropping either re-admits its own class, so no single-frame gate combination recovers the recall the stance floor spends. The recall path needs a **session- level multi-pose cue** (the board moves between poses; static clutter does not) — untestable on the single-static-capture datasets 1–5.

Operating point	Recall	Precision	Median ms	Note
Stage 1 (generator B, single-frame)	~ 2%	—	~ 82	true board, ds3 only
Stage 5 -- stance-gate	30.5%	91.6%	~ 60	full recall vs stage-4 baseline
Stage 6 -- stance-gate -- flatness 0.045*	49.3%	93.0%	~ 60	higher-recall default
Stage 5 -- strict-diamond	11.0%	100%	~ 58	~ 7:1 recall cost
Stage 8 + -- isolation 0.3	44.1%	100%	~ 63	precision-priority default

Table 1: Geometry operating points (generator B, all 535 frames). Two shipped recommendations: stage 6 for recall, stage 8 for precision. Both keep isolation default-off so stage-6 behaviour is byte-identical without the flag.

The projection + 2D scorer **core is sound**: when a clean board candidate reaches it, it produces a tight quad (2–25 mm jitter, in line with the VLP-32C noise floor) at the right place, at millisecond cost. What geometry could not do — on single static frames — is **select** that candidate reliably over board-sized clutter. That is what motivated the learned approach.

3 A Ray-Based VLP-32C Simulator

Training a segmenter needs labelled data, and the sample datasets carry no per-pixel board mask. A synthetic renderer is the only source. The Stage-9 feasibility spike found the board **is** learnable in a real range image (a coherent ~ 21-row near-range blob with a clean depth-discontinuity border) — Gate 1 **PASS** — but that a naive object-space renderer **fails the synth→ real bridge** (Gate 2 **FAIL**): sampling a regular grid in surface (u, v) coordinates and re-binning into image pixels shoots the synthetic range image through with vertical-stripe **aliasing** on every surface. A CNN would learn the artifact, not the board. The fix — and the substantial new component this phase built — is a real ray-based simulator.

3.1 Sensor model

`boarddet.sim.sensor` parses the vendored `VeloView-VLP-32C.yaml` for the 32 per-laser `vert_correction` (elevation) and `rot_correction` (azimuth-offset) pairs and generates one unit ray **per real beam angle** × azimuth step. Rows are ranked by **elevation** (0 = lowest beam, 31 = highest), not by the interleaved `laser_id` order `VeloView` records. Because every range-image pixel **is** a cast ray, there is no object-space grid-vs-image-bin step for a moiré pattern to come from — the aliasing is gone by construction.

3.2 Analytic primitives and ray casting

`boarddet.sim.primitives` provides three analytic shapes with vectorized ray-intersection: **Rect** (planar rectangle — boards, panels, walls), **Box** (oriented box), and **Cylinder** (finite side surface, no end caps — poles/pillars). `raycast.render` takes the nearest finite hit per ray, clips to the sensor’s min/max range, then applies **gaussian range noise** and **dropout** — a base random rate plus a **grazing-incidence** term that rises as the incidence cosine → 0 (dropout is evaluated against the **pre-noise** geometry, so it models sensor/surface physics, not the injected noise).

3.3 The shared range-image renderer (the train/eval linchpin)

The single most important structural guarantee: `sim.range_image.to_range_image` is the **one** renderer used for **both** synthetic and real data. Every point (sim or real) is assigned to its **nearest real channel** by elevation and to a centred azimuth bin by `atan2(y, x)`. For synthetic data the row is **exact** by construction (a ray’s true elevation **is** `sensor.elevations[row]`); rebinning a sim frame’s own points reproduces its range image up to a small constant **per-row** column shift (never a per-ray shuffle). This makes “sim and real share a row axis”

a hard, load-bearing guarantee rather than an approximation — a synth-trained model cannot fail on real data for a dumb structural reason. (This was a review-flagged linchpin that a first cut got wrong by using uniform elevation bins for the real renderer.)

3.4 Domain-randomized scene generation

scenegen.py builds ground + walls + 1–3 diamond calibration boards (a mix of plain and hollow) plus clutter, with domain randomization tuned to the deployment reality:

- Boards are **plain diamonds facing-with-tilt** (never edge-on), with $\geq 70\%$ vertical laser coverage, non-overlapping (zero cross-board mask overlap enforced), weighted 0/1/2/3 counts **including empty negatives**.
- Genuine 45° diamonds ($\sqrt{2}$ -diagonal), a $\sim 2.4\%$ zero-pixel rate from legitimate wall occlusion that the loader must handle.
- Diverse free-standing and embedded clutter (panels, boxes, cylinders).

3.5 Fidelity: the aliasing fix, verified

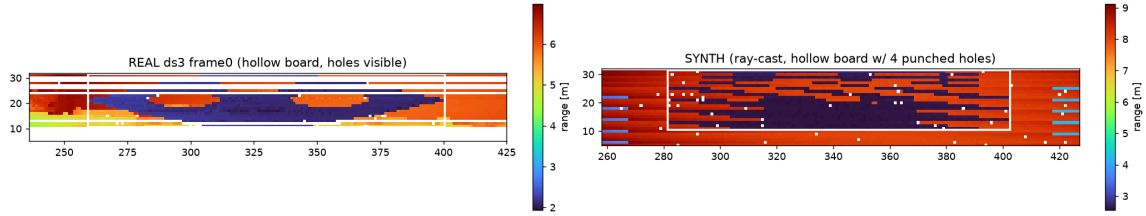


Figure 2: Gate-2 re-test: a **synthetic** range image (ray-cast along the real 32 beam angles) beside a **real** dataset-3 frame. The board is a coherent near-range blob with a clean depth-discontinuity border in both; the vertical-stripe aliasing that sank the naive object-space renderer is gone. Structural match confirmed by eye — the simulator unblocks the CNN path.

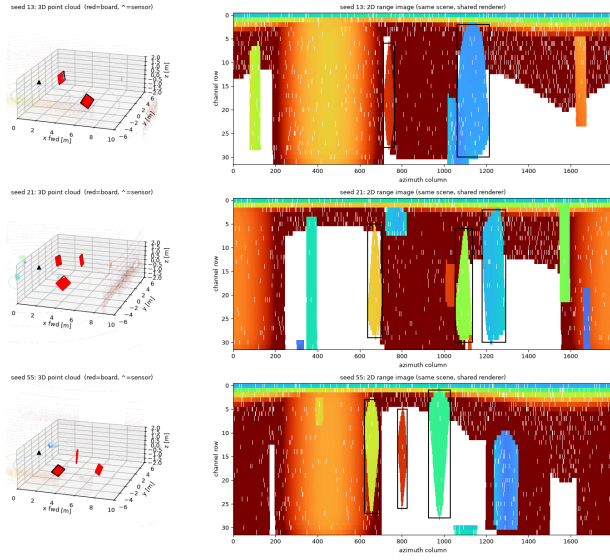


Figure 3: 3D synthetic point cloud (top) vs its 2D range-image projection (bottom) — the representation the CNN actually consumes.

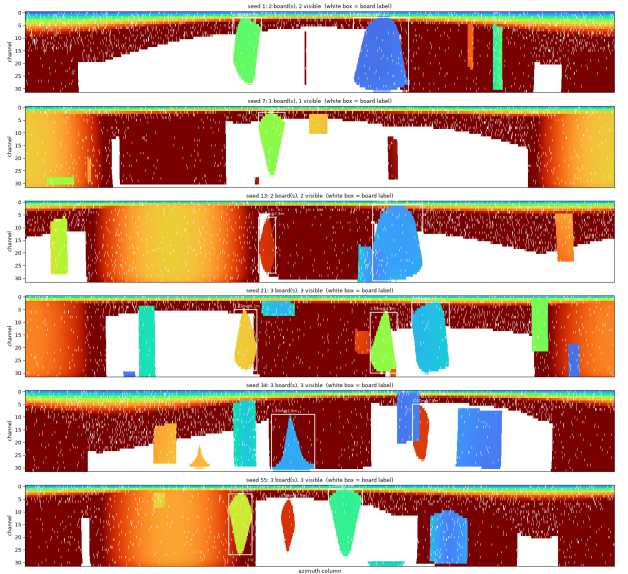


Figure 4: Six domain-randomized synthetic scenes: plain diamond boards (0–3 per scene, including empty negatives) among free-standing and embedded clutter.

4 The CNN Detector

4.1 Architecture: BoardUNet

A deliberately lightweight U-Net — **211 553 parameters** — takes a $(3, 32, W)$ range image and emits a per-pixel board-probability logit $(1, 32, W)$. Its job is **selection** (is this pixel part of an isolated, board-shaped blob?); geometry does **pose** from the pixels it selects. Three design choices matter:

- **Circular-width padding.** Azimuth wraps at the $0/2\pi$ seam, so a board straddling column 0 must see the same neighbourhood as one anywhere else. `CircularWidthConv2d` pads the **width** axis circularly and the **height** (elevation) axis with zeros — there is no laser above the top channel or below the bottom one, so wrapping height would be physically wrong.
- **Gentle vertical pooling.** With only 32 rows the encoder halves height just twice ($32 \rightarrow 16 \rightarrow 8$) while width halves a third time, so the bottleneck keeps real elevation resolution.
- **Dilated bottleneck** (dilation 2 then 4) for the wide azimuthal context an isolation-style judgment needs — a pixel must “see” far enough around it to tell whether its blob is isolated (board) or continues into a wall (clutter), which a local 3×3 receptive field cannot.

The three input channels (identical normalization for sim and real, $R_{\text{MAX}} = 12 \text{ m}$) are: (0) normalized range, (1) validity mask, (2) normalized left-neighbour discontinuity. Encoder channels $16 \rightarrow 32 \rightarrow 64$; Dice+BCE loss; logits out (sigmoid at inference).

4.2 Training and synthetic validation

Trained from scratch on the domain-randomized synthetic distribution (a 50/50 plain/hollow mix — the real eval board is hollow, the deployment target plain), on an RTX 5090. Synth/real input parity is **structurally** guaranteed by the shared renderer and normalization. The best checkpoint essentially **solves** the synthetic task:

Model	Val IoU	Pixel precision	Pixel recall
v1 (Task 32)	0.9898	0.9964	0.9934
v2 (enriched clutter, Task 36)	0.9679	0.9840	0.9834

Table 2: Synthetic held-out validation. Train/val loss track together throughout (no overfitting); enriching the clutter distribution did not make the synth fit meaningfully harder.

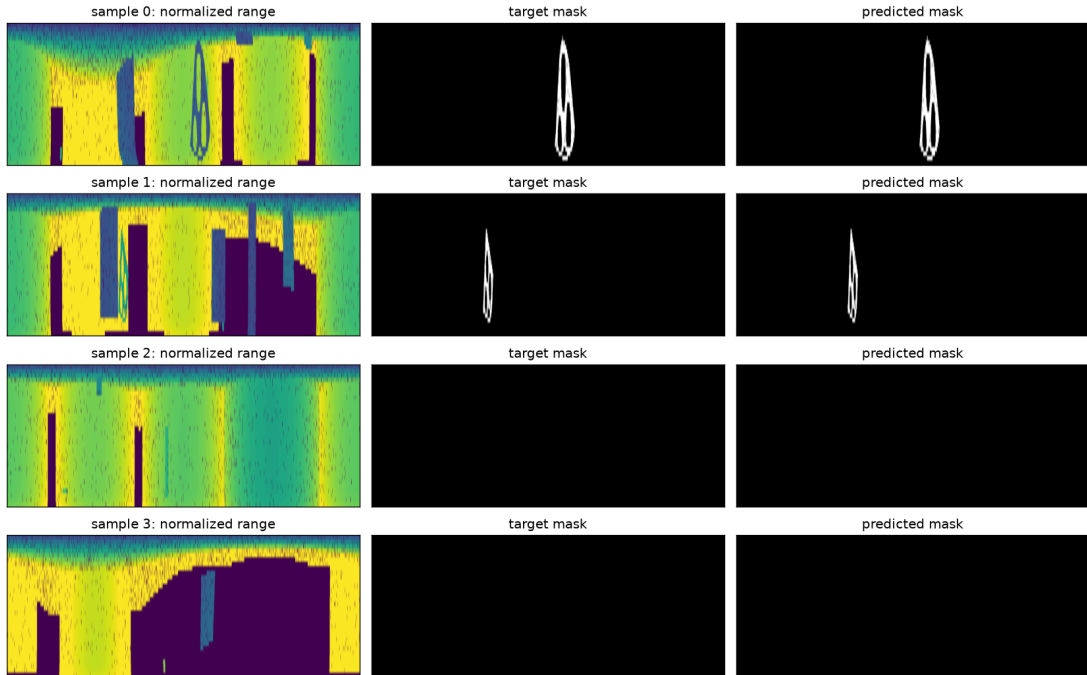


Figure 5: Synthetic validation predictions: the board mask is segmented cleanly, clutter is rejected, and empty (negative) scenes map to empty masks.

4.3 Synth → real transfer: the decisive test

`cnn.eval` runs every one of the 535 real frames through `real_frame_to_input` (the identical normalization) → `checkpoint` → `sigmoid` → `threshold` → seam-wrapping connected components → back-projection against a once-per-frame rebin of the frame’s own 3D points → `fit_fixed_square` pose → `bbox` classification (the same in/out rule geometry used).

Headline (v1): the CNN breaks the recall ceiling. Synth-trained BoardUNet on 535 **real** frames — recall **99.3% (531/535)** vs. geometry’s 44–49% ceiling — **more than 2×**. Synth→ real transfer **worked** for detection, uniformly across all 5 datasets (99–100% each). But precision was only **14.8%**: the mask fires **confidently** (sigmoid saturated, threshold-invariant) on a small, consistent set of real static fixtures the synthetic clutter never modelled.

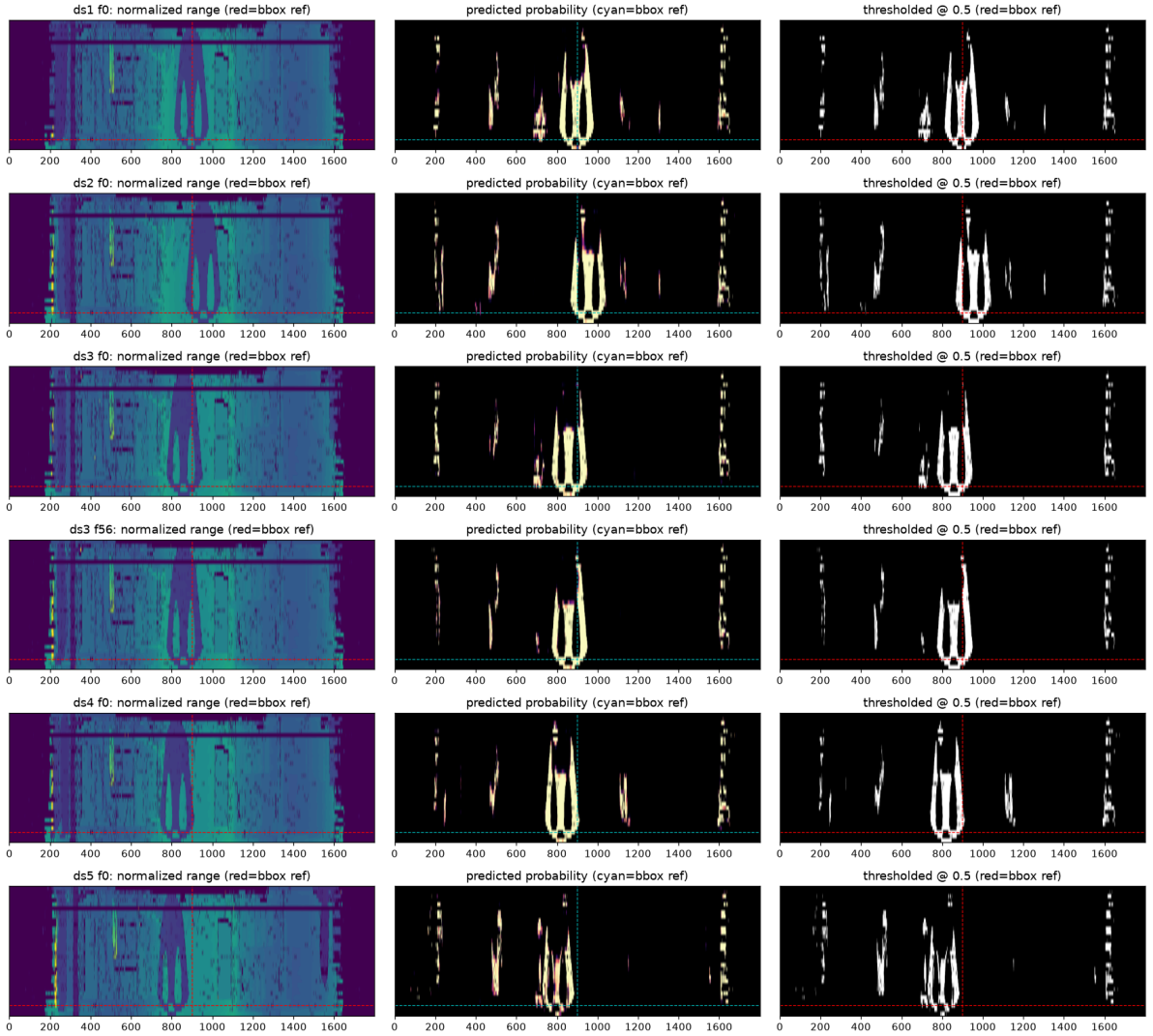


Figure 6: CNN v1 on real frames (input range image, predicted probability, thresholded mask; bbox reference marked). The board’s diamond “twin-spoke” silhouette lights up **exactly** at the bbox marker in every frame (recall transferred) — but so do several persistent real fixtures (a vertical stripe near column ~ 200 , blobs near ~ 500 and ~ 1600), with no discriminator downstream of the raw mask to catch them.

The forward pass is trivial (1.52 ms median on GPU); the ~ 89 ms post-processing cost is entirely CPU-side per-component fitting, scaling with the ~ 16 detections/frame the low precision implies. The diagnosis, reported straight: this is a **synthetic-clutter-coverage gap**, not a back-projection or pose-fitting bug — detections are computed directly from wherever the mask fires, and the mask genuinely did not learn to discriminate these fixtures from a board.

5 Hybrid Attempt and the Precision Fix

5.1 Why the obvious hybrid failed

The research-backed two-stage pattern — CNN **proposes**, geometry **verifies** — was the obvious next move: run stage 8’s isolation gate on the CNN’s candidate components. It **failed** (Task 34, NO-GO). Isolation is a **wall-embedding** detector, but the CNN’s dominant false positives are **free-standing** board-lookalikes (density 0), so isolation passes them; projected combined precision maxed $\sim 22\%$. Stage 8’s clean separation had held only because the geometry **scorer** pre-filtered clutter to wall-panels; the CNN hands isolation a broader, free-standing clutter population it is blind to by construction. Characterization (Task 34b) confirmed **no** geometric metric (size, stance, square-residual, isolation) separates the broad clutter from the board: 28% of the clutter is small scatter (poles/brackets, 0.2–0.5 m) but 72% is **large** (median diagonal 1.71 m, bigger than the board). A verifier was ruled out.

5.2 The fix: enrich the synthetic clutter, retrain

The reframe: the CNN’s false positives are a **training-data coverage gap**, and the cheapest fix (single forward pass, no inference-time geometry) is to close it in the simulator. Task 35 added three free-standing distractor kinds to scenegen.py — small **scatter clusters** (0.1–0.3 m panels, 0.03–0.1 m poles), **large panels/boxes** (0.75–1.5 m panels, up to ~ 4 m diagonal), and **non-square, arbitrarily-oriented** variants — moving the clutter- diagonal distribution from **0.58–2.24 m** (nothing under 0.5 m) to **0.21–4.08 m**, now bracketing the diagnosed 0.2–1.71 m real-FP range end to end. **Board generation was left byte-identical** (the linchpin). Task 36 retrained the **same** architecture with the **same** hyperparameters on the new default distribution and reran the **identical** eval pipeline — zero eval-side or inference- time change.

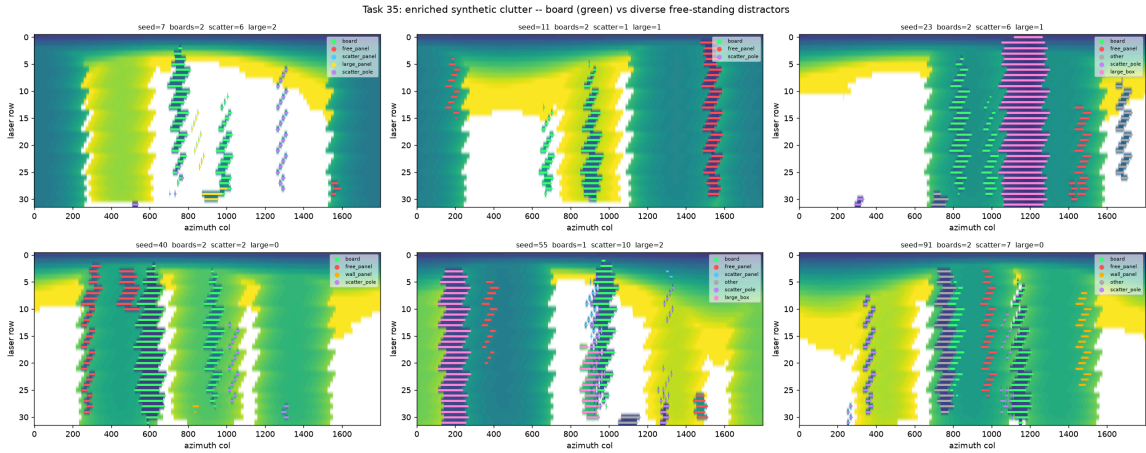


Figure 7: Enriched synthetic clutter (Task 35): diverse free-standing distractors — small scatter (poles/brackets) and large panels/boxes, in varied orientations — added alongside the original embedded panels to cover the real free-standing fixtures the v1 CNN misfired on.

5.3 Result: v1 $\rightarrow$ v2

Threshold	v1 recall	v1 precision	v2 recall	v2 precision
0.3	99.3%	14.5%	99.3%	56.1%
0.5	99.3%	14.8%	99.3%	60.8%
0.7	99.3%	15.0%	99.3%	66.1%

Table 3: Real-data eval, all 535 frames, v1 vs v2. Precision 15% $\rightarrow$ 66% (threshold 0.7) at **zero recall cost** (531/535 unchanged) — a single retrain, no eval-side code.

Stage	v1 median	v1 p95	v2 median	v2 p95
forward (GPU)	1.52 ms	1.62 ms	2.76 ms	4.22 ms
post-proc (CPU)	88.71 ms	113.76 ms	27.21 ms	49.19 ms
total	90.25 ms	115.33 ms	30.19 ms	52.40 ms

Table 4: Timing (threshold 0.5, RTX 5090). Post-processing cost tracks detection count (v1 ~ 16.4/frame, v2 ~ 3.8/frame), so cutting clutter cut CPU cost $3\times$ v2’s p95 now clears the 100 ms budget with headroom.

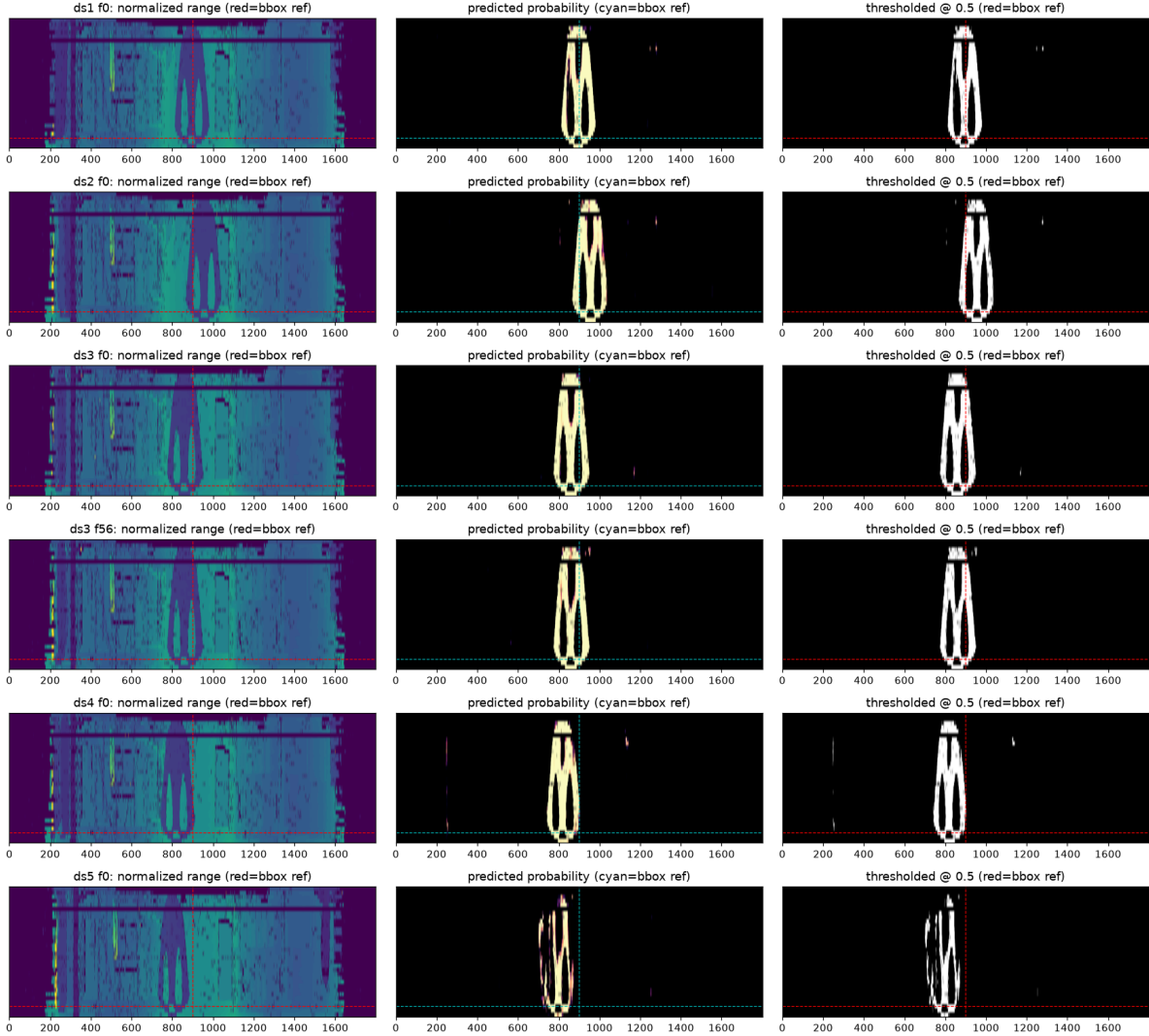


Figure 8: CNN v2 (enriched clutter) on the same 6 real frames as v1. Three of v1’s four persistent fixtures (stripe ~ 200, blob ~ 500, cluster ~ 1600) are gone or reduced to a stray pixel; the board silhouette still lights up at the bbox marker (recall held). One fixture (~ 1150–1200, ds4) persists — matching the residual-FP finding.

Residual FPs are the same zones, far sparser. Clustering the out-of-bbox detections at threshold 0.7: v1’s 7364 detections (13.8/frame) form multi-metre density-chained “mega- clusters”; v2’s 647 (1.2/frame — a **91% reduction**) form 13 tight, compact clusters, and $\geq 81\%$ of residual volume sits **inside** zones v1 also fired on — the same real-world clutter, ~ 90% suppressed, not a new failure mode. **Coverable, not structural.**

6 Consolidated Results

Configuration	Recall	Precision	Median ms	Basis
Geometry stage 6 (- - stance-gate - - flatness 0.045)	49.3%	93.0%	~ 60	264 / 20 of 535
Geometry stage 8 (+ - - isolation 0.3)	44.1%	100%	~ 63	236 / 0 of 535
CNN v1 (synth-trained, thr 0.5)	99.3%	14.8%	90.3	531 / 535; 1299 dets
CNN v2 (enriched clutter, thr 0.7)	99.3%	66.1%	30.2	531 / 535; 1264/1911

Table 5: The phase in one table. Recall is measured over 535 frames; CNN precision is over per-detection counts (multiple components per frame possible). CNN v2 median timing is at threshold 0.5 (30.2 ms); the 66.1% precision figure is at threshold 0.7.

The four rows tell the whole story. **Geometry** built a precise, well-discriminated detector whose recall is capped near half the frames by a structural selection limit. The **CNN** shattered that recall cap (2×) with a single cheap forward pass, at first paying for it with catastrophic precision. **Enriching the simulator** — the same lever that unblocked the CNN in the first place — recovered 4.4× of that precision (15% → 66%) at zero recall cost and made the detector 3× faster. What is **not** yet true: the CNN’s 66% precision remains well below geometry’s 93–100%. This is a strong partial, not a close-out.

7 Discussion and Future Work

What the arc proves. Two independent lines converged on the same fact: a sparse, ring-gapped VLP-32C board is hard to **select** over compact, board-sized clutter from a **single static frame**. Geometry hit that wall as a precision/recall trade at ~ 44–49% recall; the CNN hit it as a **precision** gap at 99% recall. Neither is a coding defect — both are data/selection limits, and the honesty of the negative results (accumulation collapsed recall; the fixed-size fitter regressed on every axis; the obvious CNN→ isolation hybrid failed) is what makes the positive results trustworthy.

The three open levers, in order of promise:

1. **Further clutter enrichment / hard-example mining.** The v1→ v2 jump (15%→ 66% from one modest retrain, with `n_scatter/n_large` frequency knobs left deliberately low) strongly suggests the residual 34% is another coverage gap in the **same** zones, not a ceiling. A round targeted at the specific remaining fixtures — or simply raising the distractor frequency — is the cheapest next step and the one most likely to move precision toward geometry’s range.
2. **A session-level multi-pose cue.** The residual clutter for **both** geometry and the CNN is **static room structure**. The calibration board is the object that **moves** between poses within a session; the fixtures never do. Requiring a detection to appear at a location that changes across poses (and down-weighting any that repeats unchanged) is the one cue that is **absent from the clutter by construction**. It is a capture-protocol change (record ≥ 2 board positions per session) untestable on the single-static-capture datasets 1–5, but the buffered multi-pose mode `advanced_extrinsic_solver` already assumes makes it a natural fit. Stages 5–8 independently converged on this as the real precision **and** recall closer.
3. **A composite CNN + geometry pipeline.** `fit_fixed_square` already hands each CNN component a pose; running geometry’s discriminators on those components is appealing but **is not** the naive isolation hybrid that failed (isolation is blind to free-standing clutter). A useful composite needs a discriminator characterized against the CNN’s **actual** free-standing FP population — more design work than a clutter-enrichment round, and lower-priority given how well the cheap retrain worked.

Integration. The projection + 2D scorer core is sound and could be ported (Rust/ROS) today for the geometry operating points, but any integration plan must budget the multi-pose/session filter as a **named, near-term** follow-on — it is the only tested path to closing the residual precision gap for either method. Generator B is the only geometry candidate generator worth carrying forward; A and C were not revisited after stage 1.

8 Conclusion

Phase 7 set out to detect a ~ 1 m calibration board anywhere in a VLP-32C cloud with no crop box and no human intervention. It delivered:

- A **validated geometry detector** (plane-fit $\rightarrow$ 2D orthographic projection $\rightarrow$ quad scorer, generator B) with two honestly-priced single-frame operating points — 49.3%/93.0% (recall-priority) and 44.1%/100% (precision-priority) — and a precise diagnosis of its ~ 44 –49% **structural recall ceiling**.
- A **faithful ray-based VLP-32C simulator** that casts the sensor’s real 32 beam angles, sharing one range-image convention with real data — the component that made a learned approach possible.
- A ~ 0.21 M-parameter **U-Net** that, trained purely on synthetic data, reaches **99.3% recall on real frames** — breaking the geometry recall ceiling by $2\times$ — and, after a single clutter-enrichment retrain, **66.1% precision** at zero recall cost and $3\times$ lower latency.

The result is not “solved”: 66% precision is a $4.4\times$ improvement over the CNN’s first attempt but still short of geometry’s 93–100%, and the recall ceiling that motivated the whole CNN detour remains structural for single-frame geometry. What the phase **did** establish is a clear, cheap, and honest path forward — one forward pass gives $\sim 99\%$ recall / 66% precision today, the eight geometry stages built the precision tools, and the remaining gap is diagnosed as coverable (more synthetic clutter) rather than fundamental. The next lever — whether more clutter enrichment or the session-level multi-pose cue every geometry stage pointed at — is well-scoped, not speculative.